

Vehicle System

Java program - Abstraction, abstract classes, final methods

Program 4: Vehicle System (Abstraction, final methods)

This file combines a runnable program (Test Cases 1,2,3,4,5,9,10) with three separate snippets (Test Cases 6, 7, 8) that each redeclare Vehicle/Car/Bike/Main to demonstrate compile-time errors. Each part was compiled and run independently below.

Source Code

```
abstract class Vehicle{
    String vehicleNumber;
    String brand;
    Vehicle(String vehicleNumber, String brand){
        this.vehicleNumber=vehicleNumber;
        this.brand=brand;
    }
    abstract void startEngine();
    final void showVehicleIdentity(){
        System.out.println("Vehicle Number:"+vehicleNumber);
        System.out.println("Brand:"+brand);
    }
}

class Car extends Vehicle{
    Car(String vehicleNumber,String brand ){
        super(vehicleNumber,brand);
    }
    void startEngine(){
        System.out.println("Car Engine starts with a key/button");
    }
}

class Bike extends Vehicle{
    Bike(String vehicleNumber, String brand){
        super(vehicleNumber,brand);
    }
    void startEngine(){
        System.out.println("Bike engine starts using a self-start/kick");
    }
}

public class Main{
    public static void main(String[]args){
        //-----Test case 1-----
        System.out.println("-----Testcase 1----");
        Car c1=new Car("KA01AB1234","Toyota");
        c1.showVehicleIdentity();
        System.out.println();
        //-----Test case 2-----
        System.out.println("----Testcase 2----");
        Bike b1=new Bike("KA05XY5678","Honda");
        b1.showVehicleIdentity();
        System.out.println();
        //-----Test case 3-----
        System.out.println("----Testcase 3----");
        //Implimentation of abstract method
        c1.startEngine();
        b1.startEngine();
        System.out.println();
        //-----Test case 4-----
        System.out.println("----Testcase 4----");
        Car c2=new Car("KA12RG6746","Renault");
        c2.showVehicleIdentity();
        Bike b2= new Bike("KA45dh3456","Honda");
        b2.showVehicleIdentity();
        System.out.println();
        //-----Test case 5 constructor initialization
        System.out.println("----Testcase 5----");
        Car c3=new Car("KA09CD9876","Hyundai");
        c3.showVehicleIdentity();
        System.out.println();
        //-----Test case 9-----invalid vehicle Number
```

```

System.out.println("----Testcase 9----");
Bike b3=new Bike("", "Jupiter");
if(b3.vehicleNumber.isEmpty()){
    System.out.println("Invalid Vehicle Number");
}
else{
    b3.showVehicleIdentity();
}
System.out.println();

//-----Test case 10-----Empty brand name
System.out.println("----Testcase 10----");
Car c4=new Car("KA67HB3478", "");
c4.showVehicleIdentity();
}
}

```

Output - Test Cases 1, 2, 3, 4, 5, 9, 10

Output

----Testcase 1----

Vehicle Number:KA01AB1234

Brand:Toyota

----Testcase 2----

Vehicle Number:KA05XY5678

Brand:Honda

----Testcase 3----

Car Engine starts with a key/button

Bike engine starts using a self-start/kick

----Testcase 4----

Vehicle Number:KA12RG6746

Brand:Renault

Vehicle Number:KA45dh3456

Brand:Honda

----Testcase 5----

Vehicle Number:KA09CD9876

Brand:Hyundai

----Testcase 9----

Invalid Vehicle Number

----Testcase 10----

Vehicle Number:KA67HB3478

Brand:

Test Case 6 - Instantiating an Abstract Class

Separate snippet: attempts 'new Vehicle()' directly.

Source Code

```
abstract class Vehicle{
    String vehicleNumber;
    String brand;
    Vehicle(String vehicleNumber, String brand){
        this.vehicleNumber=vehicleNumber;
        this.brand=brand;
    }
    abstract void startEngine();
    final void showVehicleIdentity(){
        System.out.println("Vehicle Number:"+vehicleNumber);
        System.out.println("Brand:"+brand);
    }
}

public class Main{
    public static void main(String[]args){
        System.out.println("----Testcase 6----");
        Vehicle v=new Vehicle();
    }
}
```

Output

```
----Testcase 6----
Main.java:17: error: Vehicle is abstract; cannot be instantiated
    Vehicle v=new Vehicle();
                ^
1 error
```

(Compile-time error: an abstract class cannot be instantiated directly.)

Test Case 7 - Missing Abstract Method Implementation

Separate snippet: Car does not implement startEngine().

Source Code

```
abstract class Vehicle{
    String vehicleNumber;
    String brand;
    Vehicle(String vehicleNumber, String brand){
        this.vehicleNumber=vehicleNumber;
        this.brand=brand;
    }
    abstract void startEngine();
    final void showVehicleIdentity(){
        System.out.println("Vehicle Number:"+vehicleNumber);
        System.out.println("Brand:"+brand);
    }
}

class Car extends Vehicle{
    Car(String vehicleNumber,String brand ){
        super(vehicleNumber,brand);
    }
}

class Bike extends Vehicle{
    Bike(String vehicleNumber, String brand){
        super(vehicleNumber,brand);
    }
    void startEngine(){
        System.out.println("Bike engine starts using a self-start/kick");
    }
}

public class Main{
    public static void main(String[]args){
```

```
        System.out.println("----Testcase 7----");
        Car c=new Car("KA56DF3425","Toyota");
    }
}
```

Output

----Testcase 7----

Main.java:14: error: Car is not abstract and does not override abstract method startEngine() in Vehicle

class Car extends Vehicle{

^

1 error

(Compile-time error: a concrete subclass must implement all abstract methods.)

Test Case 8 - Overriding a Final Method

Separate snippet: Car tries to override the final showVehicleIdentity() method.

Source Code

```
abstract class Vehicle{
    String vehicleNumber;
    String brand;
    Vehicle(String vehicleNumber, String brand){
        this.vehicleNumber=vehicleNumber;
        this.brand=brand;
    }
    abstract void startEngine();
    final void showVehicleIdentity(){
        System.out.println("Vehicle Number:"+vehicleNumber);
        System.out.println("Brand:"+brand);
    }
}
class Car extends Vehicle{
    Car(String vehicleNumber,String brand ){
        super(vehicleNumber,brand);
    }
    void startEngine(){
        System.out.println("Car Engine starts");
    }
    void showVehicleIdentity(){
        System.out.println("Vehicle Identity:");
    }
}
class Bike extends Vehicle{
    Bike(String vehicleNumber, String brand){
        super(vehicleNumber,brand);
    }
    void startEngine(){
        System.out.println("Bike engine starts using a self-start/kick");
    }
}
public class Main{
    public static void main(String[]args){
        System.out.println("----Testcase 8----");
        Car c=new Car("KA56DF3425","Toyota");
    }
}
```

Output

----Testcase 8----

Main.java:21: error: showVehicleIdentity() in Car cannot override showVehicleIdentity() in Vehicle

void showVehicleIdentity(){

^

overridden method is final

1 error

(Compile-time error: a method declared 'final' cannot be overridden.)